

JD Pi515

GIVING YOUTH THE {CODE} TO SUCCEED



JOHN DEERE

JOHN DEERE
CONFIDENTIAL



str = “Strings”

index, slice, methods



str = “Strings”

index, slice, methods



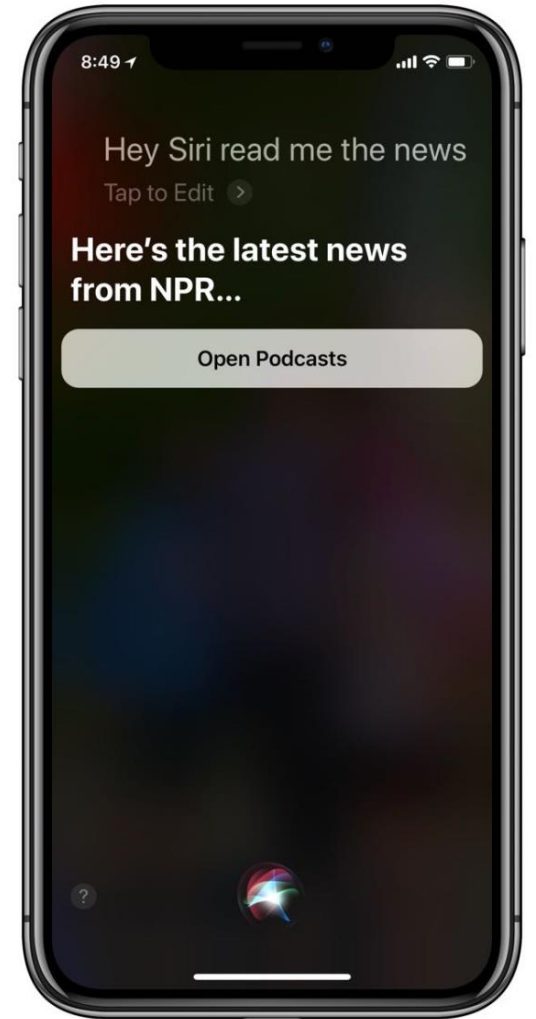
Terminology

String

An ordered series of characters or symbols

Strings are the basis for computer science challenges like natural language processing (NLP) and computational genetics

ex. Siri, ChatGPT, DigitizeMyDocs (Ryan's old team)



Terminology

Index

A number (key), which provides access to a value within a string (lock).

Indices (plural)

In computing, indices usually start at 0. **'0th index'**

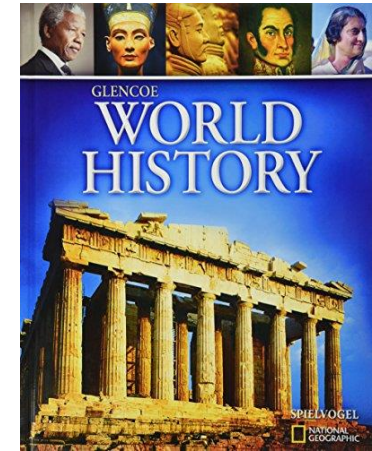


Real Life Indices

Addresses[**123 Street St**] =



Library[**123-4-56-789012-3**] =



Phones[**515-123-4567**] =



string[index]

Individual String Facts

The `len()` function tells us the number of characters in a string

```
len('MyString')  
=> 8
```

The `in` keyword tells us if one string can fit inside the other

```
's' in 'string'  
=> True
```

```
'z' in 'string'  
=> False
```


Terminology

Keyword

Words that are reserved solely for the purpose of special programming syntax within a certain language. Keywords cannot be used as identifiers like variable names.

ex. `in` = 'not allowed'

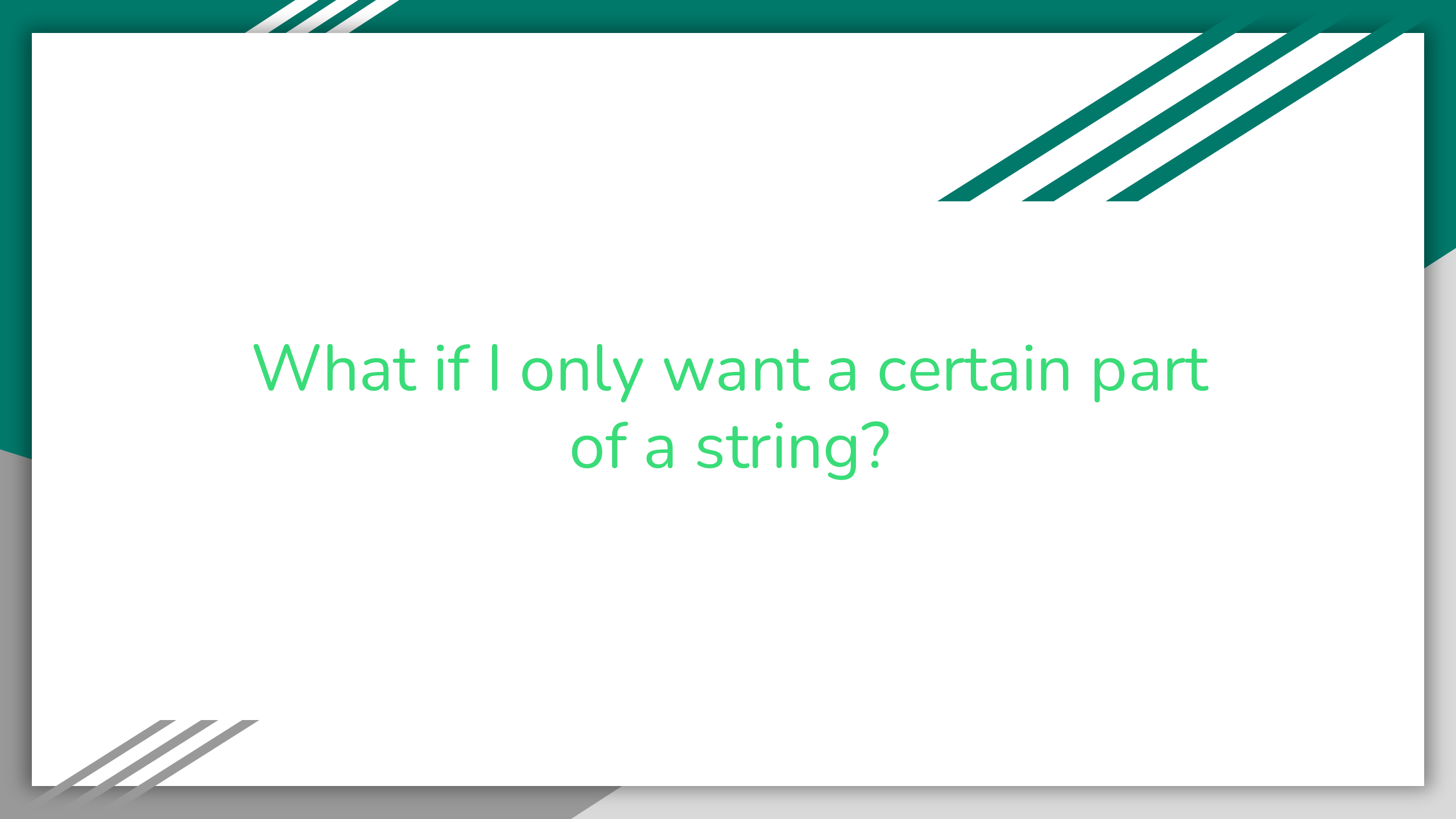


secretString Exercise



NOICE
job





What if I only want a certain part
of a string?

Terminology

Substring

A string that exists within another string

ex. 'Sand' - 'and'

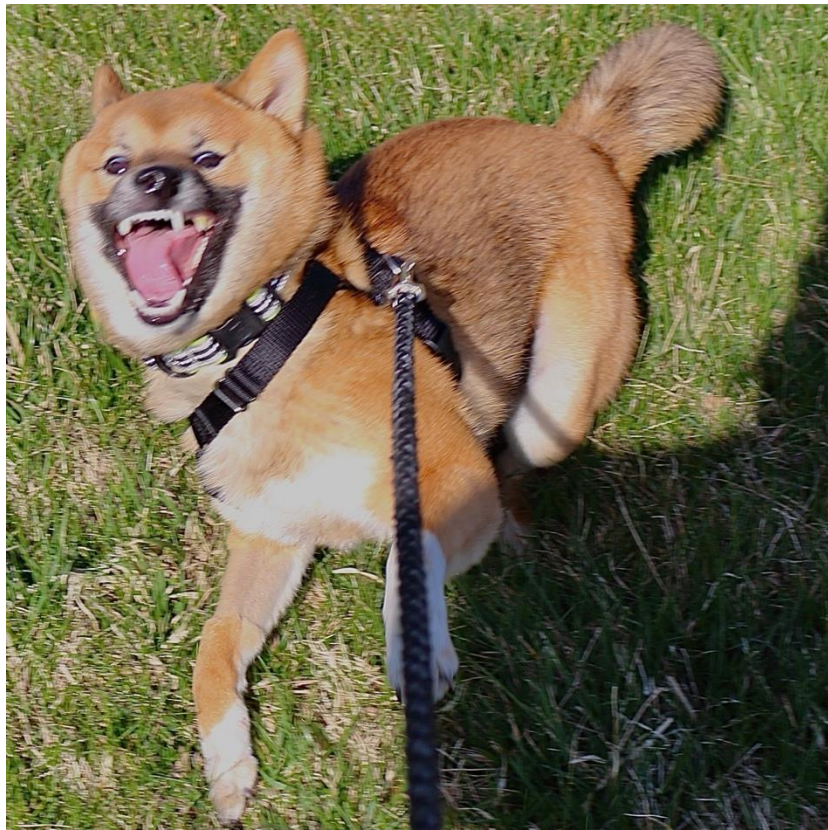
ex. 'Something' - 'some', 'thing'

ex. 'Noice' - 'no', 'ice'

Exercise 3

Use index accessors to print out the substring “Some” from the larger string “Something”

NOICE Job





What if there's an easier way?



Introducing Slices

Remember math class? Domains? Ranges?

x is defined on domain $[0,3]$...

Python kind of has its own version of this called slices.

string[start : stop]

Exercise 3b

Use slices to print out the substring “Some” from the larger string “Something”



NOICE
job



Translating Slice Syntax

“Everything from 0 to 4 except 4”

`‘Something’[0:4]`

`‘Some’`

Slices Save Time

Imagine writing this except with a string from 0 to 100

```
a = 'Something'[0] # 'S'  
b = 'Something'[1] # 'o'  
c = 'Something'[2] # 'm'  
d = 'Something'[3] # 'e'
```

More Slice Stuff

What happens if we leave out parts of the domain?

`'Something'[:4]`

`'Something'[0:]`

What happens if we use negative numbers?

`'Something'[0:-1]`

`'Something'[-1:-4]`

Negative Indices Explained

Negative indices are really just regular indices with an implied length expression

```
ex = "Nolce"  
ex[0:-1]  
ex[0:len(ex) - 1]  
ex[0:5 - 1]  
ex[0:4]
```

Negative Indices Explained

Negative indices are really just regular indices with an implied expression with the string length

```
ex = "Nolce"  
ex[-3:]  
ex[len(ex) - 3:]  
ex[5 - 3:]  
ex[2:]
```

Programming Terminology

Expression

Code that can go on the right side of the assignment operator ('=')

Sometimes you can *inline* an *expression* into other syntax, which can remove the need for an extra variable

```
example[0:len(example) - 1]
```




NOICE
job



String Methods

String Methods

All strings have functions called *methods* that can be accessed by using the *dot operator* ('.'). These *methods* allow us to handle strings in complex ways.

Terminology

Method

Methods are functions that exist as properties on certain variables, data, or objects in a programming language.



String.method(parameters)

Exercise 5

Capitalize your **firstName**

```
firstName.capitalize()
```

Make your **firstName** all uppercase

```
firstName.upper()
```

Make your **firstName** all lowercase

```
firstName.lower()
```

'Nolce' to 'Noice' in one line

```
'Nolce'.lower().capitalize()
```




A pawper gentleman

NOICE
job



Terminology

Chaining

Sequential access of properties or methods originating from the same variable, data, or object. This programming pattern can be more readable sometimes.

```
myVariable  
  .filter(unwanted)  
  .sum()
```

```
'Nolce'  
  .lower()  
  .capitalize()
```


Terminology

Immutable



Once a string is initialized or returned from a method, the string is considered constant and *immutable*. Directly changing strings is forbidden in Python.

```
nolce = 'Nolce'  
nolce[2] = 'i'    # ...???
```

```
'Nolce'  
    .lower()  
    .capitalize()
```

```
# memory – 'Nolce'  
# 'noice'  
# 'Noice'
```

Terminology

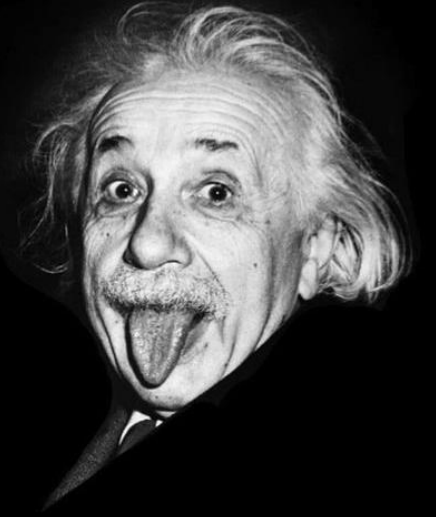
Idempotent

no matter how many times you execute something, you always get the same result for the same input

This concept will become **very important** in mathematics, computer science, and especially in **software engineering** (i.e. API development).

"Insanity is doing the same thing over and over again and expecting different results"

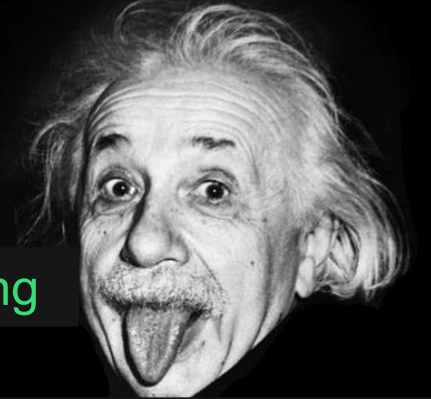
Albert Einstein



Terminology

Idempotent

"sanity is doing the same thing over and over again and always getting the same results"



- Literally everyone maintaining production software

no matter how many times you execute something, you always get the same result for the same input

This concept will become **very important** in mathematics, computer science, and especially in **software engineering (i.e. API development)**.

Split

`string.split(delimiter)`

`'Make like a banana and split!'.split('banana')`

method breaks apart the string anywhere that the `delimiter` is present and returns the resulting substrings in a list
⇒ `['Make like a ', ' and split!']`

Terminology

Delimiter

A character, symbol, or *substring* that indicates the beginning or end of data units within a larger sequence.

Important to understand for data manipulation with formats like .csv files.

Join

delimiter.**join**(subStrings)

'banana'.**join**(['Make like a ', ' and split!'])

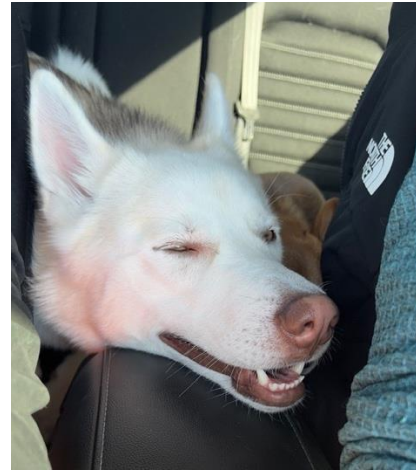
takes a list of substrings and glues them together => 'Make like a banana and split!'
using a *delimiter*

Replace

string.`replace`(search, replacement)

replaces all instances of
<search> with <replacement>
in the original string

'Make like a banana and split!'
 `.replace('banana', 'pear')`
=> 'Make like a pear and split!'



NOICE
job





